

Phase 0 Linux — Oracle Cloud VM Setup

Developer: Daniel Boone | github.com/dboone323/HavenWorld

Game Server: [https://\[your-oracle-ip\].nip.io](https://[your-oracle-ip].nip.io)

Date: September 18, 2026 | **Estimated Time:** 2-4 hours

Prerequisites: Complete Part 1 (Phase 0 Mac) before starting these steps. All steps tagged **[LINUX]** are performed on the Oracle Cloud ARM VM via SSH from your MacBook Pro.

Table of Contents

Section	Topic	Steps
4.1	Oracle Cloud VM Provisioning	A1-A5
4.2	Server Hardening and Base Configuration	B1-B6
4.3	Install Node.js, pnpm, and Git	C1-C7
4.4	Install and Configure PostgreSQL	D1-D5
4.5	Install and Configure Redis	E1-E2
4.6	Install Nginx and Configure Reverse Proxy with Free SSL	F1-F6
4.7	Install PM2 Process Manager	G1-G4
4.8	Oracle VM Quick-Reference Commands	—
4.9	Phase 0 Linux Completion Checklist	—

Section 4: Phase 0 — Linux Tasks

Every step in this section is performed on the **Oracle Cloud ARM VM** over SSH from your MacBook Pro, unless a step explicitly states otherwise. Do not skip or reorder steps — each builds on the previous.

4.1 Oracle Cloud VM Provisioning

[LINUX] LINUX-0-A1 — Sign Up for Oracle Cloud Free Tier

Go to <https://cloud.oracle.com/> and click "**Start for free.**"

You will need:

- Valid email address
- Credit card (identity verification only — no charges on Always Free resources)
- Phone number for SMS verification

HOME REGION — CHOOSE CAREFULLY

Your Home Region **cannot be changed after account creation.**

Recommended: **US East (Ashburn)** for US-based players. Approval is usually instant but can take up to 24 hours.

[LINUX] LINUX-0-A2 — Create the Free ARM VM Instance

In the Oracle Cloud dashboard: **Compute** → **Instances** → **Create Instance**. Follow each field exactly:

Step	Field	Value
1	Instance Name	havenworld-server
2	Change Image	Ubuntu 22.04 LTS
3	Change Shape	Ampere ARM → VM.Standard.A1.Flex → 4 OCPUs + 24 GB RAM

Step	Field	Value
4	Add SSH Keys	Paste both keys (one per line): personal Mac key + GitHub Actions deploy key
5	Boot Volume	100 GB custom size
6	Create	Click Create , wait 2–3 min for Running state
7	Copy IP	Copy the Public IP Address — used in every step that follows

To retrieve your SSH keys on Mac before pasting:

```
# Personal key cat ~/.ssh/id_ed25519.pub # Deploy key cat
~/.ssh/havenworld_deploy.pub
```



NOTE — Write Down Your IP

Oracle VM Public IP: _____

[LINUX] LINUX-0-A3 — Open Required Ports in Oracle Cloud Security Rules

Navigate to: **Networking** → **Virtual Cloud Networks** → **your VCN** → **Security Lists** → **Default Security List** → **Add Ingress Rules**.

Rule	Source CIDR	Protocol	Port	Description
1	0.0.0.0/0	TCP	80	HTTP
2	0.0.0.0/0	TCP	443	HTTPS
3	0.0.0.0/0	TCP	22	SSH (may already exist)

Click **Save Changes**.

[LINUX] LINUX-0-A4 — Connect to the VM from Mac via SSH

```
ssh -i ~/.ssh/id_ed25519 ubuntu@[YOUR_ORACLE_PUBLIC_IP]
```

Type yes if asked about host authenticity. Your prompt changes to:

```
ubuntu@havenworld-server:~$
```

[LINUX] LINUX-0-A5 — Add the GitHub Actions Deploy Key to the VM

While connected to the Oracle VM:

```
echo "PASTE_CONTENTS_OF_havenworld_deploy.pub_HERE" >> ~/.ssh/authorized_keys
cat ~/.ssh/authorized_keys
```

You should see **exactly two lines** — your personal key and the deploy key.

4.2 Server Hardening and Base Configuration

[LINUX] LINUX-0-B1 — Update All System Packages

```
sudo apt update && sudo apt upgrade -y sudo apt autoremove -y
```

Takes 3-5 minutes on first run. Wait for the prompt to return before proceeding.

[LINUX] LINUX-0-B2 — Configure UFW Firewall

```
sudo ufw allow OpenSSH sudo ufw allow 80/tcp sudo ufw allow 443/tcp sudo ufw
enable sudo ufw status
```

Expected: Status `active` with ports 22, 80, and 443 showing `ALLOW`.

[LINUX] LINUX-0-B3 — Fix Oracle VM iptables Rules



WARNING — REQUIRED FOR ALL ORACLE ARM VMs

Oracle ARM VMs block ports 80 and 443 at the OS level even after UFW is configured. This is a known Oracle Cloud issue. These steps are **mandatory** — skipping them will cause Nginx to appear broken.

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo netfilter-persistent save
```

If `netfilter-persistent` is not installed:

```
sudo apt install -y iptables-persistent sudo netfilter-persistent save
```

[LINUX] LINUX-0-B4 — Create a Dedicated App User

```
sudo adduser havenworld --disabled-password --gecos "" sudo usermod -aG sudo
havenworld sudo mkdir -p /home/havenworld/.ssh sudo cp ~/.ssh/authorized_keys
/home/havenworld/.ssh/authorized_keys sudo chown -R havenworld:havenworld
/home/havenworld/.ssh sudo chmod 700 /home/havenworld/.ssh sudo chmod 600
/home/havenworld/.ssh/authorized_keys
```

[LINUX] LINUX-0-B5 — Set Server Timezone

```
sudo timedatectl set-timezone America/Chicago timedatectl
```

Verify output shows CDT or CST.

[LINUX] LINUX-0-B6 — Harden SSH Configuration



CRITICAL — Read Before Proceeding

Keep your current SSH session open until you verify a brand-new session works after restarting sshd. A misconfiguration here can lock you out of the VM permanently.

```
sudo nano /etc/ssh/sshd_config
```

Set or add these exact lines:

```
PermitRootLogin no PasswordAuthentication no PubkeyAuthentication yes  
MaxAuthTries 3
```

Save: **Ctrl+O** → **Enter** → **Ctrl+X**, then restart SSH:

```
sudo systemctl restart sshd
```

Open a **NEW Mac Terminal tab** and test the connection:

```
ssh -i ~/.ssh/id_ed25519 ubuntu@[YOUR_IP]
```

Only close the original session after the new connection succeeds.

4.3 Install Node.js, pnpm, and Git

[LINUX] LINUX-0-C1 — Switch to the havenworld User

All remaining steps use the `havenworld` user unless otherwise noted:

```
sudo -u havenworld -i
```

Prompt changes to: `havenworld@havenworld-server:~$`

[LINUX] LINUX-0-C2 — Install nvm and Node.js 22 LTS

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |  
bash source ~/.bashrc nvm install 22 nvm use 22 nvm alias default 22
```

Verify:

```
node --version      # → v22.x.x npm --version      # → 10.x.x
```

[LINUX] LINUX-0-C3 — Install pnpm

```
npm install -g pnpm pnpm --version      # → 9.x.x
```

[LINUX] LINUX-0-C4 — Install Git

```
sudo apt install -y git git --version git config --global user.name  
"HavenWorld Deploy" git config --global user.email "dboone9@msn.com"
```

[LINUX] LINUX-0-C5 — Set Up GitHub Access from the Oracle VM

Generate a VM-specific deploy key:

```
ssh-keygen -t ed25519 -f ~/.ssh/github_vm_key -C "oracle-vm-havenworld" -N ""  
cat ~/.ssh/github_vm_key.pub
```

Copy the output. In GitHub: **Repo Settings** → **Deploy Keys** → **Add deploy key** →
Title: `Oracle VM` → paste key → Allow write access: **NO** → Add key.

Configure SSH to use this key for GitHub:

```
nano ~/.ssh/config
```

Add:

```
Host github.com      HostName github.com      User git      IdentityFile  
~/.ssh/github_vm_key
```

[LINUX] LINUX-0-C6 — Clone the Repository onto the Oracle VM

```
sudo mkdir -p /opt/havenworld sudo chown havenworld:havenworld  
/opt/havenworld cd /opt/havenworld git clone  
git@github.com:dboone323/HavenWorld.git .
```

Type `yes` when asked about GitHub host authenticity.

[LINUX] LINUX-0-C7 — Install Dependencies

```
cd /opt/havenworld pnpm install --frozen-lockfile
```

TIP

Run `pnpm install --frozen-lockfile` again after every `git pull` to keep dependencies in sync with the lockfile.

4.4 Install and Configure PostgreSQL

[LINUX] LINUX-0-D1 — Install PostgreSQL 16

```
sudo apt install -y postgresql postgresql-contrib sudo systemctl enable  
postgresql sudo systemctl start postgresql sudo systemctl status postgresql
```

Expected: `active (running)`

[LINUX] LINUX-0-D2 — Create Database and Application User

```
sudo -u postgres psql
```

Inside the psql shell, run each line:

```
CREATE USER havenworld WITH PASSWORD 'CHOOSE_A_STRONG_PASSWORD'; CREATE
DATABASE havenworld_prod OWNER havenworld; GRANT ALL PRIVILEGES ON DATABASE
havenworld_prod TO havenworld; \q
```

⚠ IMPORTANT — Password Security

Use a real strong password of **20+ characters**. Save it immediately — you will need it for the `DATABASE_URL` environment variable in the next section.

[LINUX] LINUX-0-D3 — Configure PostgreSQL Authentication

```
sudo nano /etc/postgresql/*/main/pg_hba.conf
```

Find the line:

```
local all all peer
```

Change to:

```
local all all md5
```

Save and restart:

```
sudo systemctl restart postgresql
```

[LINUX] LINUX-0-D4 — Test the Database Connection

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1
```

Enter your password. Success shows: `havenworld_prod=#`

Type `\q` to exit.

[LINUX] LINUX-0-D5 — Create the Production .env File

```
nano /opt/havenworld/apps/server/.env
```

Paste and fill in all values:

```
NODE_ENV=production PORT=3000 CLIENT_URL=https://havenworld.pages.dev
DATABASE_URL=postgresql://havenworld:YOUR_DB_PASSWORD@127.0.0.1:5432/
havenworld_prod REDIS_URL=redis://:YOUR_REDIS_PASSWORD@127.0.0.1:6379
JWT_ACCESS_SECRET=PASTE_FROM_GITHUB_SECRET
JWT_REFRESH_SECRET=PASTE_FROM_GITHUB_SECRET_DIFFERENT_VALUE
JWT_ACCESS_EXPIRES=15m JWT_REFRESH_EXPIRES=7d
RESEND_API_KEY=re_YOUR_KEY_HERE EMAIL_FROM=onboarding@resend.dev
SENTRY_DSN=https://YOUR_DSN@sentry.io/YOUR_PROJECT_ID ALPHA_INVITE_ONLY=true
```

Save, then lock file permissions:

```
chmod 600 /opt/havenworld/apps/server/.env
```

4.5 Install and Configure Redis

[LINUX] LINUX-0-E1 — Install Redis 7

```
sudo apt install -y redis-server sudo systemctl enable redis-server sudo  
systemctl start redis-server sudo systemctl status redis-server
```

Expected: active (running)

[LINUX] LINUX-0-E2 — Configure Redis for Security and Performance

```
sudo nano /etc/redis/redis.conf
```

Find and modify each setting (use `Ctrl+W` to search):

Setting	Change
<code>bind 127.0.0.1 ::1</code>	Verify already set — no change needed
<code>requirepass</code>	Uncomment and set: <code>requirepass CHOOSE_REDIS_PASSWORD</code>
<code>maxmemory</code>	Add line: <code>maxmemory 512mb</code>
<code>maxmemory-policy</code>	Add line: <code>maxmemory-policy allkeys-lru</code>

Save and restart:

```
sudo systemctl restart redis-server
```

Test the connection:

```
redis-cli -a CHOOSE_REDIS_PASSWORD ping # → PONG
```

Update `.env` with your Redis password:

```
REDIS_URL=redis://:CHOOSE_REDIS_PASSWORD@127.0.0.1:6379
```

4.6 Install Nginx and Configure Reverse Proxy with Free SSL

[LINUX] LINUX-0-F1 — Install Nginx

```
sudo apt install -y nginx sudo systemctl enable nginx sudo systemctl start  
nginx sudo systemctl status nginx
```

Expected: active (running)

Test: Open `http://[YOUR_ORACLE_IP]` in your Mac browser → should display the *"Welcome to nginx!"* default page.

[LINUX] LINUX-0-F2 — Understand the nip.io Domain

nip.io converts any IP address into a free domain name by replacing dots with dashes:

```
IP:      129.146.88.42 Domain:  129-146-88-42.nip.io
```

No signup, no cost, no DNS configuration needed. Let's Encrypt requires a domain name (not a bare IP) to issue SSL certificates — nip.io provides that for free.

Test: Open `http://[YOUR-IP-WITH-DASHES].nip.io` in browser → same Nginx welcome page.



NOTE — Write Down Your nip.io Domain

nip.io domain: _____

Production API URL: `https://` _____

[LINUX] LINUX-0-F3 — Install Certbot and Obtain Free SSL Certificate

```
sudo apt install -y certbot python3-certbot-nginx sudo certbot --nginx -d YOUR-IP-WITH-DASHES.nip.io
```

When prompted:

- Enter your email address
- Agree to Terms of Service: **Y**
- Skip EFF email: **N**
- Select option **2** — Redirect HTTP to HTTPS

Success message: *"Successfully received certificate."*

Verify: open `https://YOUR-IP-WITH-DASHES.nip.io` in Mac browser → padlock icon should be visible.

[LINUX] LINUX-0-F4 — Configure Nginx as Reverse Proxy for Node.js

```
sudo nano /etc/nginx/sites-available/havenworld
```

Paste this complete configuration — replace `YOUR-IP-WITH-DASHES` throughout:

```
# Redirect HTTP to HTTPS server {      listen 80;      server_name YOUR-IP-
WITH-DASHES.nip.io;      return 301 https://$host$request_uri; } # HTTPS +
WebSocket proxy to Node.js server {      listen 443 ssl;      server_name YOUR-
IP-WITH-DASHES.nip.io;      ssl_certificate /etc/letsencrypt/live/YOUR-IP-
WITH-DASHES.nip.io/fullchain.pem;      ssl_certificate_key
/etc/letsencrypt/live/YOUR-IP-WITH-DASHES.nip.io/privkey.pem;      add_header
```

```
X-Frame-Options SAMEORIGIN always;      add_header X-Content-Type-Options
nosniff always;      add_header Referrer-Policy strict-origin-when-cross-
origin always;      location / {      proxy_pass http://127.0.0.1:3000;
proxy_http_version 1.1;      # WebSocket upgrade headers (CRITICAL for
Socket.io)      proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";      proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;      proxy_set_header X-
Forwarded-For $proxy_add_x_forwarded_for;      proxy_set_header X-
Forwarded-Proto $scheme;      # Long timeout for persistent WebSocket
connections      proxy_read_timeout 3600s;      proxy_send_timeout
3600s;      keepalive_timeout 3600s;      } }
```

Enable the site and test:

```
sudo ln -s /etc/nginx/sites-available/havenworld /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default sudo nginx -t sudo systemctl reload
nginx
```

Expected from nginx -t: *"syntax is ok" and "test is successful"*

[LINUX] LINUX-0-F5 — Verify the HTTPS Proxy Chain

```
curl https://YOUR-IP-WITH-DASHES.nip.io/health
```

Expected Result at This Stage: 502 Bad Gateway — This is Correct

502 means Nginx is running and forwarding requests to Node.js, which hasn't started yet. This endpoint will return 200 after the Part 4 server setup is complete.

[LINUX] LINUX-0-F6 — Set Up Automatic SSL Certificate Renewal

Let's Encrypt certificates expire every 90 days. Certbot installs a renewal timer automatically.

```
sudo systemctl status certbot.timer
```

Test with a dry run:

```
sudo certbot renew --dry-run
```

Expected: *"Simulating renewal of an existing certificate"* with no errors.

If the timer is not active:

```
sudo systemctl enable certbot.timer && sudo systemctl start certbot.timer
```

4.7 Install PM2 Process Manager

[LINUX] LINUX-0-G1 — Install PM2 Globally

```
npm install -g pm2
```

PM2 keeps Node.js running 24/7, auto-restarts on crash, and starts automatically on VM reboot.

[LINUX] LINUX-0-G2 — Configure PM2 to Start on VM Boot

```
pm2 startup
```

PM2 prints a command beginning with `sudo env PATH=...` — **copy and run that exact command**. This registers PM2 as a systemd service so it survives reboots.

[LINUX] LINUX-0-G3 — Create PM2 Ecosystem Configuration

```
nano /opt/havenworld/ecosystem.config.js
```

Paste:

```
module.exports = {  apps: [{    name: 'havenworld-server',    script: 'dist/index.js',    cwd: '/opt/havenworld/apps/server',    instances: 1,    autorestart: true,    watch: false,    max_memory_restart: '2G',    env_file: '/opt/havenworld/apps/server/.env',    error_file: '/var/log/havenworld/error.log',    out_file: '/var/log/havenworld/out.log',    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',  }] };
```

Create the log directory:

```
sudo mkdir -p /var/log/havenworld sudo chown havenworld:havenworld /var/log/havenworld
```

[LINUX] LINUX-0-G4 — Verify PM2 Persists After Reboot



NOTE — Do This After Part 4 Server Is Running

Complete this step only after the Node.js server is deployed and running in Part 4.

```
sudo reboot
```

Wait approximately 60 seconds, then SSH back in:

```
ssh -i ~/.ssh/id_ed25519 ubuntu@[YOUR_IP] sudo -u havenworld -i pm2 status
```

Expected: `havenworld-server` shows `online` with no manual intervention required.

4.8 Oracle VM Quick-Reference Commands

Command	Purpose
<code>pm2 status</code>	Check if game server process is running
<code>pm2 logs havenworld-server --lines 50</code>	View last 50 lines of server logs
<code>pm2 restart havenworld-server</code>	Restart server after a code update
<code>pm2 stop havenworld-server</code>	Stop server temporarily
<code>pm2 monit</code>	Real-time CPU and memory dashboard
<code>sudo systemctl status nginx</code>	Check if Nginx is running
<code>sudo systemctl reload nginx</code>	Reload Nginx config after changes
<code>sudo systemctl status postgresql</code>	Check if PostgreSQL is running
<code>sudo systemctl status redis-server</code>	Check if Redis is running
<code>redis-cli -a YOUR_PASSWORD ping</code>	Test Redis — should return PONG
<code>psql -U havenworld -d havenworld_prod -h 127.0.0.1</code>	Open PostgreSQL CLI
<code>df -h</code>	Check disk space
<code>free -h</code>	Check RAM usage
<code>sudo apt install -y htop && htop</code>	Real-time process monitor
<code>curl http://localhost:3000/health</code>	Test Node.js health endpoint directly
<code>sudo certbot renew --dry-run</code>	Test Let's Encrypt renewal
<code>cat /var/log/havenworld/error.log tail -50</code>	View last 50 error log lines
<code>git -C /opt/havenworld log --oneline -10</code>	View last 10 deployed commits

4.9 Phase 0 Linux Completion Checklist

Check every item before proceeding to Part 3 (Phase 1 Mac tasks). All boxes must be checked.

ORACLE CLOUD ACCOUNT AND VM (A1-A5)

- Oracle Cloud account created and email/phone verification completed
- ARM VM (4 OCPU, 24 GB RAM, 100 GB boot disk) provisioned and in Running state
- Oracle VM public IP address noted and saved
- Oracle security rules open for ports 22, 80, and 443
- SSH connection from Mac works without password prompt

- GitHub Actions deploy key added to VM `authorized_keys`
- `cat ~/.ssh/authorized_keys` shows exactly two keys

SERVER HARDENING (B1-B6)

- All system packages updated — `apt upgrade` completed without errors
- UFW firewall enabled — `ufw status` shows Active with ports 22, 80, 443
- Oracle iptables rules added for ports 80 and 443, saved with `netfilter-persistent`
- `havenworld` OS user created with SSH access
- Server timezone set to `America/Chicago`
- SSH hardened — `PermitRootLogin no` and `PasswordAuthentication no` in `sshd_config`
- New SSH session verified working after hardening before closing original session

NODE.JS, PNPM, GIT, AND REPOSITORY (C1-C7)

- Switched to `havenworld` user — prompt shows `havenworld@havenworld-server`
- `nvm` installed and sourced — `nvm --version` returns a version
- Node.js 22 LTS installed — `node --version` shows `v22.x.x`
- `pnpm` installed — `pnpm --version` shows `9.x.x`
- Git installed and global identity configured
- GitHub VM deploy key generated and added to GitHub repo Deploy Keys (read-only)
- `~/.ssh/config` entry created for `github.com` pointing to `github_vm_key`
- Repository cloned to `/opt/havenworld`
- `pnpm install --frozen-lockfile` completed successfully

POSTGRESQL (D1-D5)

- PostgreSQL 16 installed, enabled, and running
- `havenworld` database user created with strong password (20+ characters)

- `havenworld_prod` database created
- `pg_hba.conf` updated from `peer` to `md5` auth and PostgreSQL restarted
- `psql` connection test passed — connected to `havenworld_prod` with password
- Production `.env` created at `/opt/havenworld/apps/server/.env` with all values filled in
- `.env` permissions locked to `600`

REDIS (E1-E2)

- Redis 7 installed, enabled, and running
- `redis.conf` updated — `requirepass`, `maxmemory 512mb`, `maxmemory-policy allkeys-lru` set
- Redis restarted after config changes
- `redis-cli ping` returned `PONG`
- `REDIS_URL` in `.env` updated with Redis password

NGINX, NIP.IO, AND SSL (F1-F6)

- Nginx installed, enabled, and running
- `nip.io` domain resolves in browser (shows Nginx welcome page)
- `nip.io` domain written down
- Let's Encrypt certificate obtained for `nip.io` domain
- Nginx `havenworld` site config created with WebSocket proxy headers
- Default Nginx site removed from `sites-enabled`
- `sudo nginx -t` passes — syntax ok and test successful
- `sudo systemctl reload nginx` completed
- HTTPS loads at `https://[ip].nip.io` — padlock visible in browser
- `curl https://[ip].nip.io/health` returns `502` (Nginx running, Node.js not yet started — correct)
- Certbot auto-renewal timer active

- `sudo certbot renew --dry-run` completed with no errors

PM2 (G1-G4)

- PM2 installed globally
- `pm2 startup` run and the generated `sudo env PATH=...` command executed
- PM2 systemd service registered for autostart on reboot
- `ecosystem.config.js` created at `/opt/havenworld/` with correct paths
- `/var/log/havenworld/` directory created and owned by `havenworld` user
- (After Part 4) Reboot test completed — `pm2 status` shows server online after reboot

Series Navigation

Part	Phase	Environment	Topics
Part 1	Phase 0 Mac	MacBook Pro	Dev environment, SSH keys, GitHub repo, Cloudflare Pages, GitHub Actions secrets
Part 2 ◀ YOU ARE HERE	Phase 0 Linux	Oracle Cloud ARM VM	VM provisioning, hardening, Node.js, PostgreSQL, Redis, Nginx, SSL, PM2
Part 3	Phase 1 Mac	MacBook Pro	Phaser 3 client, avatar sprite system, tilemaps, customizer UI, pathfinding
Part 4A / 4B / 4C	Phase 1 Linux	Oracle Cloud ARM VM	Node.js server, auth, Socket.io, database, invite codes, deployment

HavenWorld Complete Build Guide | Part 2 of 4: Phase 0 Linux — Oracle Cloud VM
Setup | Daniel Boone | September 18, 2026

github.com/dboone323/HavenWorld | Internal developer documentation — not for
public distribution